

AnchorOverlap: Efficient Collective-GEMM Optimization Based on Consumer Dependencies

Anonymous Author(s)

Abstract

In tensor-parallel execution of large language models, collective communication and matrix multiplication (GEMM) repeatedly alternate on the critical path. To overlap their execution, existing methods partition intermediate data and enable fine-grained data handoffs through dependency scheduling and signaling, allowing downstream tasks to begin once portions of their inputs become available. However, these methods do not guarantee that the data needed by the consumer are produced first. Moreover, retaining every unconsumed block in staging would cause the staging footprint to grow with the workload. We present *AnchorOverlap*, a runtime execution protocol for collective-GEMM chains. The protocol determines the handoff granularity and data-production order based on the consumer’s read dependencies. In AllGather-GEMM, computation on the local shard begins immediately, and dependent tasks execute as remote row chunks arrive. In GEMM-ReduceScatter and GEMM-AllReduce, execution is organized around the output frontier that enables reduction to start as early as possible: the former prioritizes the GEMM output blocks that form this frontier, whereas the latter produces output blocks in logical order and launches recursive-doubling reduction as soon as each block is computed. In both reduction chains, these output blocks are cyclically mapped to a bounded set of symmetric staging slots within an active window. Arrival and release states coordinate data handoff, and each staging slot is reused after the consumer has finished reading from it. On NVIDIA A100 and Hygon K100_AI platforms, AnchorOverlap achieves an operator-level speedup of up to **1.40×** and end-to-end prefill speedups of 1.02–1.22× for Llama3-70B and Qwen2.5-72B. Across three GEMM-ReduceScatter window experiments, halving the active-window size relative to that used for full logical-block residency reduces the symmetric staging footprint by 50%, with only a 0.4%–0.6% increase in median latency.

Keywords: computation-communication overlap; tensor parallelism; collective communication; one-sided communication; large language models

1 Introduction

As model scale and context length continue to grow, DeepSeek-V4 [8], Kimi K3 [16], and Qwen3.8 [32] have reported trillion-parameter or million-token configurations, making deployment of these generative models on a single GPU infeasible. Tensor parallelism [35], pipeline

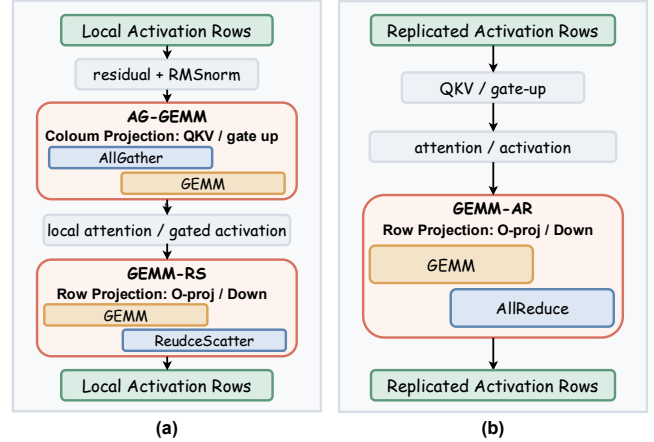


Figure 1. (a) Row-sharded activations; (b) replicated activations.

parallelism [23], and data parallelism partition parameters to support large-scale training [14] and inference serving [18, 27, 46]. The resulting collective primitives, including AllGather (AG), ReduceScatter (RS), and AllReduce (AR), repeatedly alternate with matrix multiplication (GEMM) on the critical path. Overlapping computation and communication is an important technique for mitigating this overhead. As Figure 1 shows, every Transformer layer repeatedly goes through GEMM producing partial results, collective data handoff, and downstream consumption. With row-sharded activations, the input projections in Attention and MLP obtain and consume inputs through AG-GEMM, while the output projections reduce partial results and restore the row-sharded layout through GEMM-RS. With replicated activations, the output projections instead combine partial results through GEMM-AR. In both execution paths, communication time repeatedly accumulates on the critical path. More importantly, the two ends of each chain form a closed local producer-consumer dependency: the upstream output is the downstream input, and the consumer can start once a small subset of its inputs is available rather than waiting for overall completion. This structure both creates the opportunity for overlap and defines the scope of feasible control.

Prior work overlaps dependent computation and communication through task decomposition [13, 38], kernel fusion or fine-grained dependencies [4, 45], and signals or dependency-aware scheduling [12, 31]. Overlap thus depends on handoff boundaries and task order. Across successive blocks, early consumer startup further requires two

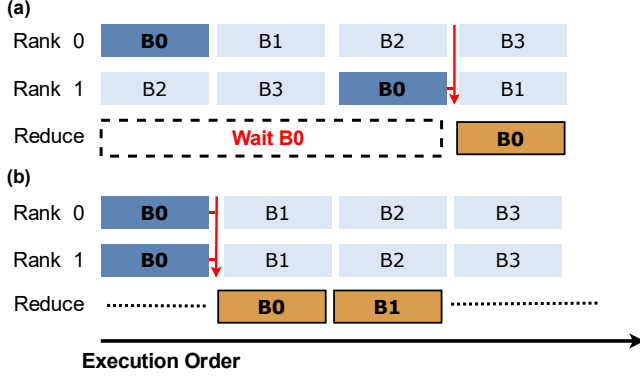


Figure 2. Effect of production order on reduction startup. (a) B0 becomes ready late. (b) Producing B0 first lets its reduction overlap earlier with the remaining GEMM work, followed by reduction of B1.

conditions: the cross-rank data the consumer needs must arrive in time, and the previous data at the receiving location must have been consumed. We take the coordination of dependency-driven data supply with storage reuse after consumption as the starting point for further improving overlap.

Starting consumers early requires answering two related questions: which data should a handoff provide, and when should that data be produced? For a selected consumer task, the handoff boundary should cover the data it actually reads. An overly coarse boundary makes the task wait for unrelated data, whereas an overly fine boundary can increase notification and scheduling overhead. Once the boundary is fixed, production order determines when these read dependencies are satisfied: some results may have signaled readiness while the data needed by the consumer remains unproduced. In GEMM-RS, for example, reducing an output block requires the corresponding contributions from the source ranks. In Figure 2(a), one source rank completes its contribution late, and readiness notifications for other outputs cannot start the selected reduction. Figure 2(b) keeps the handoff boundary unchanged but prioritizes the contributions forming the current reduction frontier, allowing the target reduction to overlap earlier with the remaining GEMM work. Handoff granularity and production order must therefore follow the same consumer dependencies to turn local data readiness into earlier consumer execution.

Sustained overlap also requires space for handing off subsequent blocks. On a one-sided communication path, data written to peer-visible staging must be retained until the relevant reads complete. Reserving a location per logical block makes the symmetric-staging footprint grow with the workload; reclaiming space only after an entire batch completes can introduce waits that disrupt sustained overlap. We therefore propose AnchorOverlap, a runtime execution protocol for collective-GEMM chains that connects execution

dependencies with staging lifetimes through data blocks that consumers can process independently. On reduction paths that cyclically reuse staging, completed consumption permits subsequent writes to the same location, sustaining execution over a bounded number of physical slots.

AnchorOverlap uses different execution strategies for the three chains in Figure 1, according to their consumer dependencies. In AG-GEMM, computation on the local shard begins immediately, and GEMM tasks that depend on remote row chunks start as those chunks arrive. In GEMM-RS and GEMM-AR, execution is organized around the output frontier that enables reduction to start early. RS prioritizes the GEMM output blocks that form this frontier, allowing reduction to overlap with computation of the remaining outputs; AR advances the reduction frontier block by block and launches recursive-doubling reduction as soon as the current output block is computed, overlapping reduction with GEMM computation for subsequent blocks. On the RS/AR paths, successive output blocks reuse symmetric staging slots within an active window. With the window depth, block shape, and number of participating devices fixed, symmetric data-staging capacity need not grow with the total number of logical blocks.

This paper makes the following contributions:

1. An execution protocol for collective-GEMM chains that bases handoff granularity and data-production order on the data consumers need to read. It is instantiated as local and remote dependency-driven execution for AG, frontier-prioritized production for RS, and per-block recursive-doubling reduction for AR.
2. A symmetric-staging reuse mechanism coupled to data handoffs. Arrival and release states govern data writes, reads, and reclamation, allowing RS/AR to execute continuously using a bounded number of physical slots.
3. We implement all three chains on two GPU platforms and use configuration search to adapt execution to different workloads. Operator- and model-level experiments evaluate the overlap benefits, while ablations examine readiness granularity, frontier-first production, and the trade-off between staging capacity and execution latency.

2 Related Work

Prior research organizes computation-communication overlap through decomposition and scheduling, fused kernels, dependency signals, and production order. We relate these approaches to AnchorOverlap by the chain stages they control.

2.1 Decomposition and Dependency-Aware Scheduling

Decomposition creates schedulable compute or tensor units and places communication between them. OCDC [39],

Domino [38], and CoCoNet [13] split dependent computation or cross compiler and communication abstractions. Centauri [5], CO2 [36], Oases [20], and Async-TP [30] tune communication partitions or training schedules. ISO [41] and TokenWeave [10] extend this idea across sequence and token dimensions, and SARATHI uses chunked prefill [2].

Syncopate [31] derives producer/consumer dependence graphs, inserts minimal synchronization, lets consumers process chunks as they arrive, and reorders waves to improve overlap across AG-GEMM, GEMM-RS, and GEMM-AR. CommFuse [15] prioritizes communication-dependent computation ahead of communication-free work to reduce the exposed tail. ConCCL [1] exploits GPU DMA engines to run communication concurrently with computation, and Lee et al. [19] characterize the performance and power implications.

These results show that overlap granularity and concurrency must be tuned per workload rather than assuming that finer granularity always performs better. If communication groups or kernel partitions do not match a consumer’s actual read set, an overly coarse handoff makes executable tasks wait for unrelated data, whereas an overly fine handoff adds notification and scheduling overhead without necessarily satisfying the consumer’s full dependencies earlier.

2.2 Fusion and Fine-Grained Kernel Dependencies

This family compiles communication and computation into the same kernel, or ties them through fine-grained dependencies, so that overlap happens inside the kernel or right at its boundaries. FLUX [4] fuses communication into GEMM kernels, while cuBLASmp [24] provides a distributed GEMM/library interface. T3 [28] tracks compute events and transparently triggers collective progress. TileLink [45] supplies tile-centric producer notification and consumer waiting, and its dependencies determine when a consumer tile may execute. Comet [43] decomposes shared tensors along consumer-consumable dimensions and prioritizes producer portions that can be used immediately by downstream MoE work. Iris [3] provides tile-based symmetric-memory abstractions and workgroup-specialized overlap in Triton.

These methods already incorporate consumer dependencies into granularity selection and execution scheduling. In collective-GEMM chains, the data a consumer needs also depends on how tensors are partitioned across ranks: completion of one tile need not satisfy all cross-rank read dependencies. With one-sided writes and bounded storage at the receiver, new data must also wait until the data previously held at the same location has been read.

2.3 Producer Signaling and Reordering

Signaling-based methods trigger communication on local completion events inside the kernel, keeping handoff close to data production. FlashOverlap [12] triggers communication using tile-completion counts within a group. It groups tiles or

waves and reorders each group’s output according to execution order into contiguous regions accepted by NCCL, restoring logical order after communication. Its grouping search balances earlier communication startup against the bandwidth benefit of larger messages, adapting communication granularity and data layout to GEMM output completion.

For the MoE return path, Cui et al. [7] combine tile-level readiness signals with segment-granular communication. A persistent GEMM kernel prioritizes tiles destined for remote ranks; a separate communication kernel issues a one-sided transfer once all tiles required by a segment are ready. The method connects production and transfer by preparing routing metadata, including destination ranks and remote offsets, in advance, and tunes segment size to balance startup time against transfer bandwidth.

FlashOverlap organizes communication groups and layout around output completion order, while Cui et al. prioritize data awaiting transfer. Together, they show how the benefit of local completion signals depends on handoff granularity and data-supply order.

2.4 Collectives and Distributed Kernel Runtimes

This category of work supplies collective algorithms, one-sided primitives, and kernel-level signal/wait scheduling. TACCL [33], MSCCLang [6], Blink [37], and AutoCCL [42] synthesize, describe, or tune collective algorithms and topology mappings. NCCL [25], NVSHMEM [26], and Triton-distributed [44] provide execution interfaces for communication, synchronization, and device-side task progress. AnchorOverlap preserves the collective semantics of AG, RS, and AR and controls when the incremental data associated with these operations is produced, consumed, and released.

Beyond data staging, GICC [34] uses monotonic epochs to recycle bounded NIC and work-queue state, while X-Stage [40] models finite post-issue capacity and sender backpressure. These mechanisms govern control state or outstanding operations.

Pipeline parallelism [22, 23] and sequence parallelism [9, 17, 21] rearrange the computation stages of training, changing where and how often these chains occur. DistServe [46] and Splitwise [27] disaggregate prefill and decoding onto separate devices. vLLM [18] and vAttention [29] improve memory management for serving. FlashDecoding++ [11] optimizes decode-time attention execution. Parallel layouts and deployment choices can change where communication occurs and how much data it moves, but these outer-level decisions alone do not specify when partial data suffices to start a consumer within an invocation, or when subsequent data may overwrite a receiving slot. Dependency-aware handoffs in each chain complement these directions.

3 System Design

AG-GEMM, GEMM-RS, and GEMM-AR differ in communication semantics but share the same producer-consumer relationship: producers generate data incrementally, and consumers can begin their next executable task as soon as the data it depends on becomes available. AnchorOverlap implements this relationship as a runtime execution protocol. This section details AnchorOverlap’s optimization mechanisms.

3.1 Overview

In the row-sharded layout of Figure 3(a), the local input enters the Local Fast GEMM Path immediately, while remote inputs trigger their dependent GEMM tasks as they arrive through AllGather. The two paths write their respective row ranges of the output tensor. Gated activation then feeds the output projection, which produces local contributions to the reduction on each rank. RS prioritizes the output blocks needed by the same reduction task. The destination can gather those contributions and start reducing them while producers compute later blocks. Each destination retains its reduced rows, yielding a row-sharded output.

In the replicated layout of Figure 3(b), every rank already holds the full activation, so the input projection requires no AllGather. After gated activation, the output projection advances the reduction frontier block by block and starts recursive-doubling reduction as soon as the current output block is computed. The reduce1 and reduce2 arrows show the two exchanges in the four-rank example. GEMM computes later blocks while the current block is reduced, and every rank eventually obtains the same complete output. In both layouts, the ready and reuse arrows mark the two repeating transitions: ready enables a completed block to be handed to its consumer, and reuse returns consumed staging space to a later block.

For a group of p ranks, A_s , W_s , and D_s denote rank s ’s activation, weight, and output tensors, and the GEMM shape is $M \times N \times K$. Row-sharded AG inputs and RS outputs contain $m = M/p$ local rows. In AR, each rank’s GEMM produces M rows of local results, and each rank retains all M output rows after reduction. C_c and B_b denote a contiguous row chunk and a column band. We call their intersection, indexed by (c, b) , a panel. Labels such as C_0/B_0 on the right of Figure 3 identify these output blocks. Table 1 summarizes the notation.

Ω denotes all logical tasks in an invocation, and $\mathcal{A}_t \subseteq \Omega$ denotes the tasks still occupying physical slots at time t . AG hands off input row chunks; RS and AR hand off output panels through reusable symmetric staging slots, releasing each slot after its consumer finishes reading.

3.2 Consumer Startup and Data-Production Order

In AG-GEMM, consumer startup depends on the input rows it reads. Tasks that depend only on the local shard can enter

Table 1. Notation used in the design.

Symbol	Meaning
p	Number of ranks in the communication group.
s, d, u	Source, destination, and remote AG ranks.
A_s, W_s, D_s	Rank-local activation, weight, and output tensors.
C_c, B_b	Row chunk and column band.
c, b	Chunk and band indices.
$h_c, w_b, w_b^{\max}$	Chunk height, band width, and maximum band width.
B	Number of column bands.
F	Number of priority row chunks per RS destination.
L	RS active-window depth.
R_{RS}, R_{AR}	RS reducer-pipeline count and AR output-block slot count.
q, r	RS window slot and AR output-block slot.
ω, x	Logical data block and its mapped physical slot.
$\gamma, \tau_\omega, \pi_\omega$	RS ready epoch; current and predecessor physical-slot tickets.

the local GEMM path immediately. Remote tasks wait for the row chunks (u, c) they actually read. A GEMM tile spanning several chunks waits for all of them, without waiting for unrelated remote inputs. Local computation can therefore overlap AllGather, and computation on arrived remote chunks can overlap the remaining transfers.

An RS consumer needs every rank’s contribution to the same output region. For example, 0/0 in Figure 3(a) identifies the first row chunk and first column band of a destination rank. Completing other regions does not make this panel reducible while one of its contributions is still missing. AnchorOverlap calls the producer tasks that complete these dependencies *wake-up work*, and the computation that continues after the consumer starts *remaining production work*. To complete these dependencies early, each producer in the multi-band frontier path computes the first F row chunks for every destination before the remaining chunks, bringing the contributions required by these reductions into the early production phase. Tiles within this frontier remain independently schedulable, and reductions start according to actual readiness, overlapping the remaining GEMM work.

First-output commit time allows us to compare startup behavior across the two implementations. Figure 4 breaks this process into producer-launch-to-ready, ready-to-consumer, and consumer-to-first-commit intervals. Across the three recorded shapes, the frontier-first implementation commits

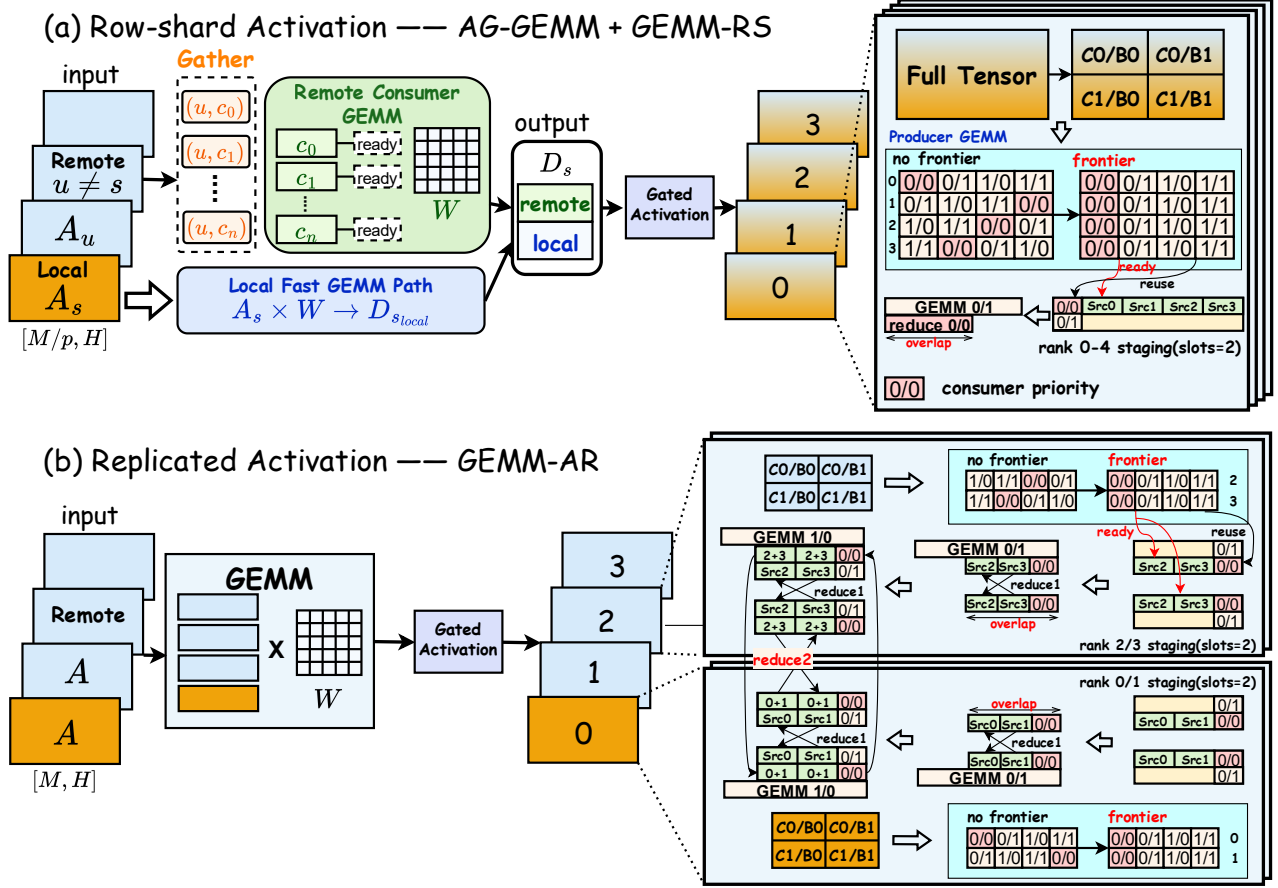


Figure 3. AnchorOverlap under two tensor-parallel layouts: (a) AG-GEMM and GEMM-RS with row-sharded activations; (b) GEMM-AR with replicated activations. Ready connects production to consumption; reuse returns consumed slots to later blocks. Reduce1 and reduce2 are the two exchanges of four-rank AR.

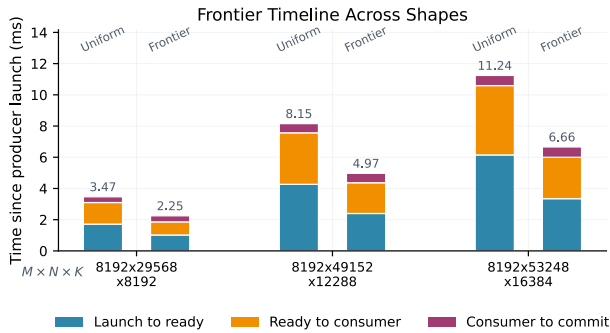


Figure 4. First-output timelines of the uniform-order and frontier-first implementations.

its first output earlier than the uniform-order implementation.

AR retains the complete reduced output on every rank. In Figure 3(b), ranks jointly advance the current frontier block

awaiting reduction. AnchorOverlap submits its reduction immediately after its production task, allowing reduction to start when the result is ready and overlap subsequent block computation. The frontier advances in the same row-chunk-major, column-band-minor order on every rank. For block 0/0, reduce1 forms two-rank sums that reduce2 combines into the four-rank result, overlapping the GEMMs for later blocks such as 0/1 and 1/0. RS prioritizes production to complete reduction dependencies; AR uses immediate per-block submission to keep ready results from waiting behind later GEMMs. Both connect startup to ongoing production.

3.3 Reclamation with Active-Windowed Symmetric Staging

RS and AR both use symmetric staging to receive data from other ranks: local contributions to the same output block in RS, and accumulated results from each recursive-doubling exchange in AR. Once all reads and output updates for the current block finish, the space can serve a later block.

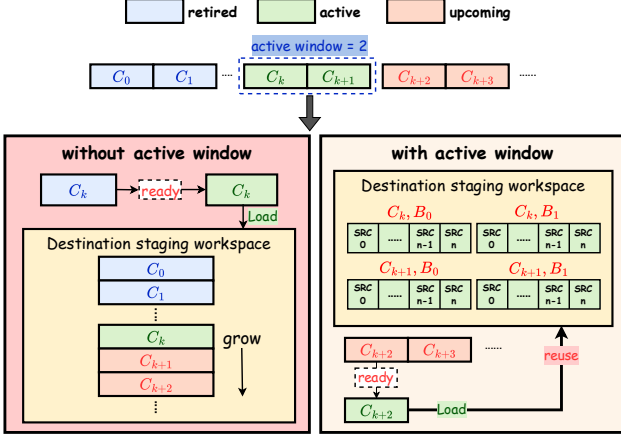


Figure 5. Staging layout and reuse with an active window, illustrated for RS. Two row-chunk positions each contain two column-band panels; each panel position can be reused after consumption.

AnchorOverlap bounds the number of tasks occupying staging slots, allowing logical blocks to advance through a reused physical allocation.

Figure 5 expands this reuse mechanism. The left side reserves a separate region per logical row chunk, so staging grows with the chunk count. The right side keeps a window of depth $L = 2$ holding C_k and C_{k+1} . Each row chunk is split into column bands B_0 and B_1 , and each panel’s SRC regions receive contributions from the respective source ranks. After a panel of C_k is consumed, the corresponding panel of C_{k+2} can use the same physical position; C_{k+1} and C_{k+3} reuse positions in the same way. Column bands determine the extent of a reduction task, while window depth determines how many row chunks have staging space at once.

RS maps row chunk c to window position $q = c \bmod L$ while retaining column-band index b , yielding LB panel positions. AR maps logical block $k = cB + b$ to $r = k \bmod R_{AR}$ and reserves R_{AR} output-block slots. In Figure 3(b), a block holds the same slot across both exchanges, using different source-rank regions within it for the incoming data of each phase. The slot becomes reusable after the second reduction phase. $\mathcal{A}_t$ changes as blocks finish consumption and new blocks enter, subject to these slot-count bounds.

Let h_c be the chunk height, $w_b^{\max}$ the maximum band width, and e the bytes per element. Allocating for this configuration requires approximately the following data-staging capacity per rank for RS:

$$M_{RS}^{\text{data}} \approx L \cdot B \cdot p \cdot h_c \cdot w_b^{\max} \cdot e. \quad (1)$$

Each panel position reserves a receiving region for each of the p source ranks. The local contribution seeds the reduction directly, saving a staging write, although the allocation retains its source region. AR likewise partitions each slot by

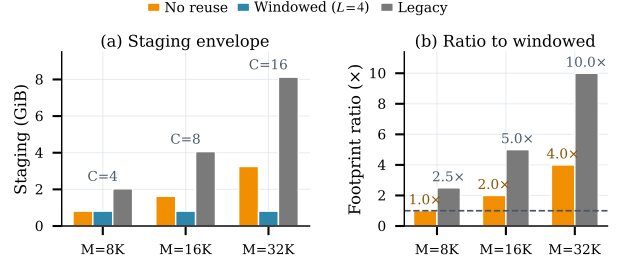


Figure 6. Symmetric-staging capacity versus logical row-chunk count at fixed $N = 53248$, $K = 16384$, and $L = 4$.

source rank and requires

$$M_{AR}^{\text{data}} \approx R_{AR} \cdot p \cdot h_c \cdot w_b^{\max} \cdot e. \quad (2)$$

Each AR phase reads the region belonging to its exchange peer. These equations describe symmetric staging for received data; local GEMM results and final outputs reside in ordinary device memory.

With block dimensions, band count, group size, and window depth fixed, increasing the logical row-chunk count requires no additional data slots. Figure 6 uses $L = 4$ to compare capacities calculated from the allocation layouts: the active-window allocation stays fixed, while a separate allocation for every logical chunk grows with the chunk count. Staging envelope in the figure denotes the symmetric staging capacity reserved for that configuration. A small window can make later transfers wait for a free position, whereas a large window increases memory usage, so window depth must also account for execution latency.

Bounded slots also require a rule for when reuse is safe. In Figure 5, ready means that the current source panel has been produced, while reuse requires the destination to have finished reading the previous panel at the same position. The transfer stream writes new data only after both conditions hold and publishes arrival after the write completes. The consumer waits for all data required by its current task, finishes its reads and output update, and then publishes release. This sequence protects unconsumed data from later panel writes.

Because a slot receives successive blocks, arrival and release states carry a sequence number for each use. The consumer waits for the current block’s arrival number, and the transfer stream waits for the preceding block’s release number at that slot. State left by an old block therefore cannot act as completion notification for a new one. RS releases a panel position after its reduction; AR releases a block slot after all of its recursive-doubling phases. This release feedback implements the reuse paths in Figure 3, sustaining production and transfer within finite staging space.

4 Operator Instantiations and Runtime Tuning

This section describes the implementation of AnchorOverlap. We combine Triton consumer kernels, one-sided communication, and symmetric-memory primitives to submit computation, transfer data, and perform reduction for each chain. Configuration search selects the handoff granularity and execution resources for a given problem shape to reduce operator latency.

4.1 AG-GEMM Instantiation

Rank s holds activation shard A_s and weight shard W_s . AllGather supplies the other activation shards, and the output is partitioned by input source:

$$D_{s,u} = A_u W_s, \quad u \in [0, p). \quad (3)$$

The local GEMM reads A_s directly and writes $D_{s,s}$. Meanwhile, AllGather copies the other ranks' inputs into a workspace, from which the remote GEMM consumes arriving row chunks and writes the corresponding $D_{s,u}$. The two computation paths use separate streams and write disjoint output rows, matching the Local Fast GEMM Path and Remote Consumer GEMM in Figure 3(a).

Algorithm 1 gives the row-chunk handoff path. InitReady zeroes the ready array on the caller stream, and the communication stream waits for this initialization. The remote consumer runs on the caller stream, while the local compute stream also waits for input preparation. A completed transfer publishes $\text{ready}[u, c] = 1$ only after the copied data is readable. Each remote GEMM tile derives its dependencies from the input rows it reads and waits for all of those chunks before computing. Ready tiles can proceed while the rest of AllGather is still running. When the communication path or consumer does not support row-chunk signals, the runtime uses rank-granular readiness.

In the algorithm, $\mathcal{G}_{\text{remote}}$ contains remote output tiles, and $\mathcal{D}(g)$ contains the input chunks read by tile g . A **submit** block enqueues work asynchronously; local GEMM, AllGather, and independent remote tiles can therefore overlap. Publish exposes input shards before they are copied, and Join makes downstream work await all three paths. Contexts are reused in invocation order.

4.2 RS/AR Execution and Slot Reuse

4.2.1 Runtime State and Task Submission. The runtime creates an execution context for each reduction chain, holding local GEMM results, output buffers, symmetric staging slots, and communication streams. The producer writes local results, transfer streams copy the data to be handed off, and reducer streams read staging and update the output. The runtime enqueues waits on the corresponding device streams and establishes event dependencies between communication and reduction. These waits require no additional

Algorithm 1: Row-chunk-driven AG-GEMM

Input: $A_s, W_s, p; \{C_c\};$ workspace X
Output: $D_s = \text{concat}_{u=0}^{p-1}(A_u W_s)$

```

1 InitReady(ready);
2 submit local GEMM do
3    $D_{s,s} \leftarrow A_s W_s;$ 
4 submit AllGather do
5    $\hat{A} \leftarrow \text{Publish}(A_s);$ 
6   foreach  $(u, c), u \neq s$  do
7      $X[u, c] \leftarrow \text{Copy}(\hat{A}_u[C_c]);$ 
8      $\text{ready}[u, c] \leftarrow 1;$ 
9 foreach  $g \in \mathcal{G}_{\text{remote}}$  do
10  submit tile  $g$  do
11    await  $\bigwedge_{(u,c) \in \mathcal{D}(g)} \text{ready}[u, c] = 1;$ 
12     $D_s[g] \leftarrow \text{TileGEMM}(X, W_s, g);$ 
13 Join();
14 return  $D_s;$ 
```

polling thread blocks. Communication and reduction use high-priority streams, with resources configured to allow production and consumption to run concurrently.

Each staging position has an arrival state $\text{arrival}_d[s, x]$ and a release state $\text{free}_d[x]$, identifying the data delivered by source s and the data consumed at destination d . To distinguish successive blocks at the same position, the AnchorOverlap runtime also maintains an invocation sequence base β , an RS readiness epoch γ , and an array $\text{last}[x]$. This array records the latest scheduled sequence number for position x , from which the next block derives its release value; it does not indicate completed consumption.

For logical block $\omega = (c, b)$, the runtime assigns $\tau_\omega = \beta + cB + b + 1$, obtains the predecessor number π_ω from $\text{last}[x]$, and updates that entry to τ_ω . For example, in an RS window of depth two, $(c + 2, b)$ reuses the position of (c, b) and therefore waits for its release number. State starts at zero when the context is created. Subsequent invocations retain the per-slot records and advance β ; RS also advances its readiness epoch γ . Ranks use matching block indices and sequence ranges, and each context is reused in invocation order.

Algorithm 2 of the supplementary material gives the source and destination operations, distinguishing asynchronous submission from completed consumption. The following instantiations show how RS and AR use these states.

4.2.2 RS Frontier Production and Windowed Reduction. The multi-band RS path implements frontier-first production with two GEMM kernels submitted in sequence. The first computes the first F row chunks for every destination rank, and the second computes the remaining chunks. Once

all output tiles of a panel have been stored, the producer publishes its `chunk_signal`. The transfer stream can then copy that panel, allowing reduction to overlap the still-running production kernels. A single-band configuration uses a producer that publishes readiness by row chunk.

Let $P_{s \rightarrow d}[C_c, B_b]$ be the local contribution generated by source rank s for destination d . The destination computes

$$D_d[C_c, B_b] = \sum_{s=0}^{p-1} P_{s \rightarrow d}[C_c, B_b]. \quad (4)$$

The source communication stream writes each remote contribution into source s 's region of the destination window and publishes $\text{arrival}_d[s, x] = \tau_\omega$. The destination reducer waits for the local panel and matching arrival numbers from all remote sources. It seeds the reduction with the local contribution, reads and adds the remote contributions, writes the output, and sets $\text{free}_d[x] = \tau_\omega$. Using the local contribution directly saves one staging copy.

AnchorOverlap assigns panels round-robin to R_{RS} reduction pipelines in handoff order. Each pipeline waits for its own task dependencies, while a window position becomes available to a later panel after consumption. Pipeline count controls concurrently advancing tasks; window depth controls resident data. They are configured separately.

4.2.3 AR Frontier Handoff and Recursive Doubling.

For each AR frontier block, AnchorOverlap submits GEMM to the compute stream and immediately submits consumption to a reducer stream, following the common block order defined in Section 3.2. The current cuBLAS path writes block GEMMs into a preallocated local-result buffer, and the reducer stream copies each result to its output accumulation region.

Reduction starts with $Z_s^{(0)} = P_s[C_c, B_b]$. For a power-of-two group size, at phase $j \in [0, \log_2 p)$ rank s exchanges its current accumulated result with $v = s \oplus 2^j$, where $\oplus$ denotes bitwise XOR:

$$Z_s^{(j+1)} = Z_s^{(j)} + Z_v^{(j)}. \quad (5)$$

After $\log_2 p$ phases, each rank holds $D_s[C_c, B_b]$, the sum of all ranks' contributions $P_u[C_c, B_b]$. The two phases in the four-rank case correspond to `reduce1` and `reduce2` in Figure 3(b). They execute sequentially and overlap later blocks' GEMMs.

For each exchange, the send stream waits for the current accumulated result and the peer slot's release, then copies the result into this rank's source region at the peer. The local reducer waits for both outgoing-copy completion and incoming data before adding in place, so it cannot modify data still being read by the send operation. After the final phase, it publishes the slot's release number and records a local block-completion event.

AnchorOverlap uses these events to limit how far production can run ahead. With a production lookahead of L_{prod} row chunks and n output blocks, the number of allowed outstanding blocks is $W_{\text{look}} = \min(R_{AR}, L_{\text{prod}}B, n)$. Before submitting

block $k \geq W_{\text{look}}$, the compute stream waits for the local completion event of block $k - W_{\text{look}}$. Release numbers continue to protect remote slots.

4.3 Constraint-Aware Configuration Search

Earlier consumer startup does not necessarily make the smallest blocks or highest communication concurrency the fastest configuration. Small blocks create more handoff opportunities but also more notifications and task submissions. More reduction resources can accelerate consumption while taking resources from GEMM. A shallow window makes transfers more likely to wait for release; a deep window increases staging usage. These tradeoffs vary with matrix shape, so AnchorOverlap selects a configuration for each shape and reuses it in subsequent invocations.

The search seeks low end-to-end collective-GEMM latency among executable candidates. For operator o , shape $x = (M, N, K, p)$, and hardware environment H , a candidate is

$$\theta_o = (h_c, w_b, \ell, \rho, \chi_o), \quad (6)$$

where h_c, w_b determine the handoff volume, ℓ, ρ control communication concurrency and consumer SM allocation, and χ_o holds chain-specific settings. RS uses frontier depth, window depth, and pipeline count; AR uses block-slot count, production lookahead, and GEMM backend. Fully measuring every combination would require many runs, so the search separates structural selection from resource refinement.

Algorithm 3 of the supplementary material gives the two-stage search. Taking RS candidate organization as an example, the searcher normalizes the relationships among window depth, frontier depth, and task count, removes duplicates, and groups candidates by (h_c, L, B) . It retains a few representatives from each bucket using a heuristic based on window-to-reduction-queue proportions, communication-lane count, and the fraction of the window occupied by the frontier. The heuristic reduces the number of candidates to measure; full-chain latency determines their performance. The run budget is divided between structural screening and resource refinement.

The first stage measures structural candidates briefly with an initial steady-state and tail SM allocation. Candidates are ranked by the maximum total latency across ranks. The second stage takes the faster structures and measures alternative steady-state and tail SM allocations, avoiding a full resource sweep for every structure. The searcher then takes the fastest successful second-stage candidates and increases the warmup and measurement counts, selecting the lowest-latency configuration among successful longer runs. If all longer runs fail, the script retains the coarse-measurement winner as a fallback without successful long-run validation.

Candidate runs include actual workspace allocation and operator execution. Candidates that fail allocation or execution are excluded from valid latency rankings. The

symmetric-memory budget requires

$$M_o^{\text{data}}(\theta) + M_o^{\text{meta}}(\theta) \leq M_{\text{sym}}^{\text{budget}}, \quad o \in \{\text{RS}, \text{AR}\}, \quad (7)$$

where the data term is the receiving storage in Equations (1) and (2), and the metadata term covers state arrays and allocation alignment. A shared preallocated workspace is charged at its reserved dimensions. This budget is a necessary feasibility condition; actual allocation and execution determine whether a candidate produces a timing record.

For AG, AnchorOverlap instead selects kernel configurations for the local GEMM and the remote consumer so that both paths suit the input shape.

5 Evaluation

We evaluate AnchorOverlap at the operator, TP-layer, and end-to-end prefill levels, followed by mechanism studies of ready granularity, frontier-first scheduling, active-windowed staging, and configuration search. All speedups are normalized to matched baselines within the same platform.

5.1 Experimental Setup

5.1.1 Platforms and Software. The primary server has four NVIDIA A100-SXM4-80GB GPUs with NVLink NV12 between every pair, running Ubuntu 24.04 (Linux 5.15) and driver 580.65.06. It runs Python 3.12, PyTorch 2.7.0 (CUDA 12.6), NCCL 2.26.2, Triton 3.4.0, and NVSHMEM 3.3.9. The second server has four Hygon K100_AI-64GB accelerators and runs DTK 26.04, Python 3.11, PyTorch 2.7.1, Triton 3.1.0, RCCL 2.22.3, and DUSHMEM 3.2.5. DUSHMEM supplies the DCU-side one-sided symmetric-memory primitives corresponding to NVSHMEM, allowing the same ready/ticket/active-window protocol semantics to be instantiated there. A100 covers all operator chains, layer integration, and the main mechanism ablations; K100_AI covers operators and model-level prefill.

5.1.2 Workloads, Baselines, and Timing. Operator experiments cover all three chains. Model-level experiments cover TP Attention, TP MLP, and end-to-end prefill for Llama3-70B and Qwen2.5-72B with TP=4, BF16, batch 64, and sequence length 128, hence $M = 8192$. The timed E2E path spans token embedding, all Transformer layers, final normalization, and the LM-head vocabulary projection, returning the logits for the last prefill token. The MLP (M, N, K) shapes are $(8192, 28672, 8192)$ and $(8192, 29568, 8192)$, respectively.

Attention and MLP use 20 warmups and the mean of 100 graph replays per launch; E2E prefill uses 10 warmups and the mean of 20 timed replays. CUDA synchronization brackets each path, an NVSHMEM barrier separates Torch and Triton segments. After warmup, CUDA events measure the replay loop; we synchronize the stream and divide the total event time by the iteration count. Compilation and graph

capture are excluded. For each model-level condition, we retain the speedup from one final launch.

Operator plots use a serialized collective-GEMM baseline implemented with PyTorch/NCCL on A100 and PyTorch/RCCL on K100_AI. Applicable comparisons are FLUX and Triton-distributed for AG; FLUX, FlashOverlap, and Triton-distributed for RS; and FlashOverlap and Triton-distributed for AR. The AG-ready study normalizes to Triton-distributed, while the RS-frontier study normalizes to the matched uniform-scheduling implementation.

The model-level prefill figure reports the speedup of each recorded path over a semantically equivalent Torch no-overlap reference:

$$\text{Speedup}_{\text{torch}} = \frac{T_{\text{torch}}}{T_{\text{variant}}}. \quad (8)$$

5.1.3 Correctness and Slot-Reuse Validation. Correctness checks and performance measurements use separate executions. Performance figures use records from configurations that pass correctness checks and complete timing successfully. Model-level Attention, MLP, and E2E checks all use $(\text{atol}, \text{rtol}) = (2 \times 10^{-2}, 2 \times 10^{-2})$. For E2E correctness, we apply FP32 softmax to the logits and compare the probabilities with a Torch/HuggingFace reference. Softmax and comparison remain outside the timed interval.

Operator checks compare the output on every rank against a semantically equivalent serialized Torch implementation; a check passes when all ranks satisfy the tolerance. Table 2 reports the per-chain tolerances and launch outcomes. To exercise repeated slot reuse, the RS window sweep uses eight logical chunks per segment and $L \in \{1, 2, 4, 8\}$. Depths 1, 2, and 4 force different chunks to reuse the same physical slot. AG checks cover ready granularities from 2048 to 256 rows.

5.2 Operator-Level Performance

Figures 7 and 8 compare operator performance across the three chains on A100 and K100_AI, respectively, normalized to each chain’s serialized baseline on the same platform.

As Figure 7 shows, the AnchorOverlap paths reach 0.98–1.16× for AG, 1.02–1.27× for RS, and 0.99–1.19× for AR over the serialized baseline across the four tested A100 shapes. As Figure 8 shows, the same shapes on K100_AI reach 1.08–1.35× for AG, 1.06–1.40× for RS, and 1.06–1.23× for AR, with the paper-wide 1.40× peak on the RS shape $(8192, 29568, 8192)$.

FlashOverlap’s tuning search cannot complete the RS and AR chains on $(8192, 29568, 8192)$: evaluating the first candidate aborts with an error on both paths, because the candidate tile shape does not divide the matrix dimensions.

5.2.1 Feasibility-Aware Comparison. The default symmetric heap of about 1 GiB/PE causes Triton-distributed to fail allocation on several RS/AR shapes. Raising the budget to 8 GiB/PE allows it to complete all four shapes. Figure 9 pairs

Table 2. Chain-level correctness and launch outcomes, including development runs. Each BF16 threshold applies to both absolute and relative tolerance.

Chain	BF16 tol.	Reuse stress and pass record
AG-GEMM	10^{-3}	121/129 launches passed; eight failures came from two early debug batches of one shape.
GEMM-RS	2×10^{-2}	Forced cyclic reuse; window sweep 60/60 passed, controlled order 11/12 completed (one numerical check passed, followed by a timing timeout).
GEMM-AR	2×10^{-2}	Forced cyclic reuse; window sweep 60/60 passed, controlled order 12/12 completed.

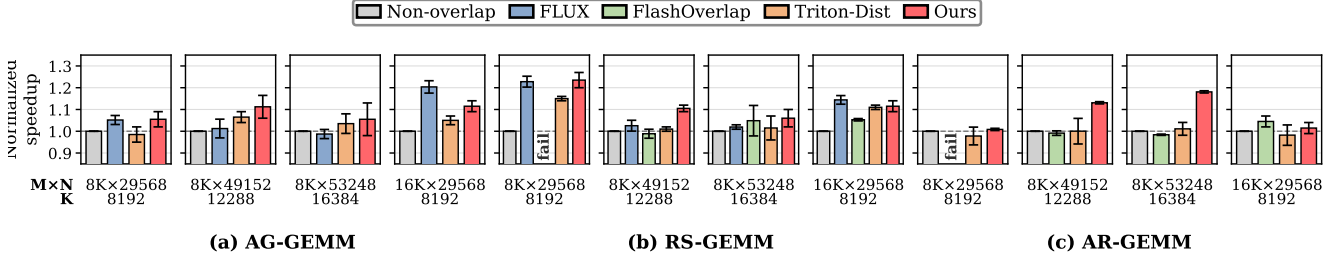


Figure 7. Operator-level speedup on four A100 GPUs, normalized separately for each collective-GEMM chain to its matched serialized baseline.

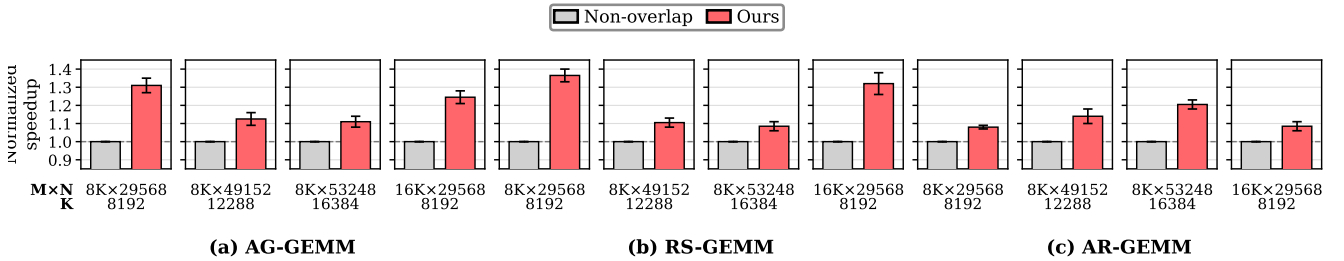


Figure 8. Operator-level speedup on four K100_AI accelerators, normalized separately for each collective-GEMM chain to its matched serialized baseline.

speedup after enlargement with symmetric-staging demand. The lines report Triton-distributed’s measured allocation and AnchorOverlap’s fixed window reservation as fractions of the heap budget. Triton-distributed’s allocation grows from 1.13 to 2.26 GiB/PE for RS and from 2.27 to 4.54 GiB/PE for AR. AnchorOverlap reserves 0.81 GiB/PE for RS and 1.63 GiB/PE for AR in these configurations, allowing all four shapes to share a fixed-capacity staging window.

To examine how the buffer demand scales with the work, we raise the symmetric-heap budget to 32 GiB and, in Figure 10, compare the symmetric-memory allocations of the two RS implementations as the matrix rows grow, with $N=53248$ and $K=16384$ fixed. If every logical unit permanently owns a symmetric-staging location, the allocation grows with the logical work set: the no-reuse baseline grows linearly from 2.03 GiB at $M=8192$, and the 32.5 GiB required at $M=131072$ exceeds the budget and fails allocation. AnchorOverlap stays at about 0.81 GiB with a fixed $L=4$ active window, decoupling the symmetric-staging footprint

from the logical work and leaving a roughly $20\times$ gap at $M=65536$.

5.3 Model-Level Prefill Evaluation

Model execution includes attention, activations, normalization, the LM head, and other work outside our collective-GEMM paths. We therefore test whether operator gains carry into TP Attention, TP MLP, and end-to-end prefill that includes the LM head, reporting both the combined AG/RS path and the AR path.

5.3.1 TP Attention, TP MLP, and End-to-End Prefill. Figure 11 contains one timing record for each model, platform, and evaluated path. On A100, the combined Ours AG/Ours RS path reaches $1.16\times/1.15\times$ for Attention, $1.13\times/1.11\times$ for MLP, and $1.05\times/1.09\times$ for end-to-end prefill on Llama3-70B/Qwen2.5-72B. Ours AR reaches $1.01\times/1.02\times$

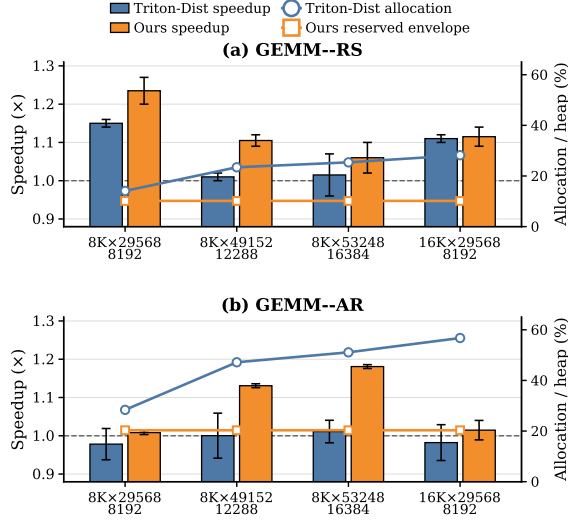


Figure 9. GEMM-RS/AR speedup and symmetric-memory allocation on A100. The right-axis percentages use an 8 GiB/PE heap budget as the denominator and correspond to Triton-distributed’s measured allocation and AnchorOverlap’s fixed staging reservation.

for end-to-end prefill. Triton-distributed and variants replacing only AG or RS show that the chains contribute differently across workloads.

On K100_AI, end-to-end prefill reaches 1.20–1.22× with Ours AG/RS and 1.09–1.10× with Ours AR.

5.4 Ablation Analysis

We use ablation studies to examine ready granularity, frontier-first scheduling, and active-windowed staging.

5.4.1 Fine-Grained Ready Units. The AG study compares rank-ready with four fixed row-tile-ready granularities using rank-max latency and speedup over Triton-distributed. Figure 12 shows three measured shapes with different communication-to-compute ratios and message sizes.

Coarse readiness delays consumer startup, while overly fine readiness can offset overlap gains with signaling, polling, and scheduling overhead. In Figure 12, the two larger- N shapes peak at 1024-row readiness (1.086× and 1.085×), while the smaller- N, K shape peaks at 256 rows (1.045×), where rank-ready even falls below the reference (0.985×). The appropriate granularity thus varies with the workload; finer granularity does not always improve performance.

5.4.2 Consumer-Anchored Frontier Scheduling. To test whether prioritizing consumer wake-up work improves startup and complete-chain latency, we compare the uniform and frontier-first GEMM-RS paths under matched shapes and runtime resources. The study uses three shapes:

(8192, 29568, 8192) is the Qwen2.5-72B TP-prefill MLP projection, while (8192, 49152, 12288) and (8192, 53248, 16384) cover wider or deeper workloads.

The frontier-first path achieves lower complete-chain latency on all three shapes (Figure 13), with speedups of 1.02–1.31× over uniform scheduling. The largest gain occurs at (8192, 29568, 8192), while (8192, 53248, 16384) gains 1.02×. These results show that supplying reduction inputs earlier can improve complete-chain latency, with the size of the gain varying across matrix shapes.

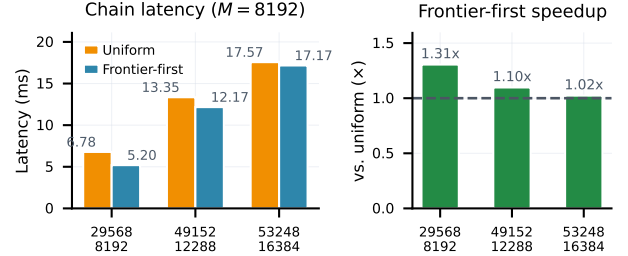


Figure 13. Matched GEMM-RS implementations with uniform and consumer-anchored frontier-first order.

5.4.3 Active-Windowed Symmetric Staging. The window study uses the same three GEMM-RS shapes, each with eight logical chunks. A depth of $L = 8$ holds all chunks simultaneously and serves as the full-residency reference. We perform five independent launches at each $L \in \{1, 2, 4, 8\}$, completing 60 launches in total.

Figure 14 reports median latency relative to full residency and the symmetric-staging footprint, comprising data slots and arrival/free ticket arrays.

A smaller window reduces resident capacity but makes later chunks more likely to wait for slots that have not yet been released. At $L = 1$, the footprint falls by 87.5%, with median latency increasing by 4.86%–20.85% across the three shapes. At $L = 2$, the footprint reduction is 75% and the latency increase is 0.67%–12.06%. At $L = 4$, the footprint is halved while median latency increases by only 0.41%–0.58%.

A half-depth window thus sustains near-full-residency performance on the three tested shapes; further capacity reductions incur a larger, shape-dependent latency cost.

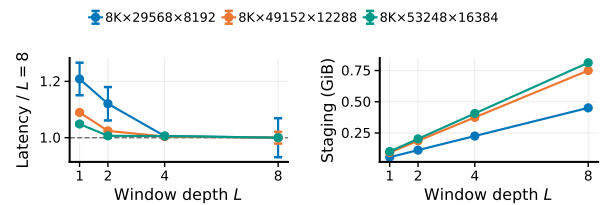


Figure 14. GEMM-RS latency-staging tradeoff across active-window depths and three case-study shapes.

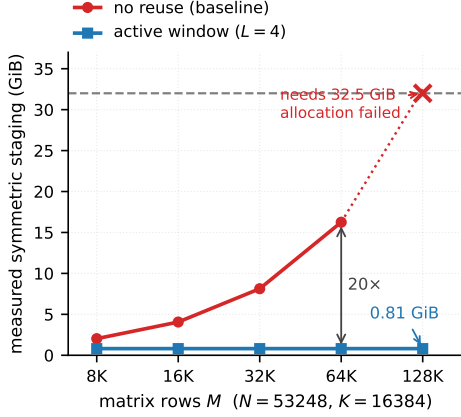


Figure 10. Symmetric-staging allocation and demand versus the matrix dimension M . With $N=53248$, $K=16384$, and a 32 GiB/PE heap cap, the no-reuse baseline requires 32.5 GiB/PE at $M=131072$ and fails allocation; the $L=4$ active window stays at 0.81 GiB/PE.

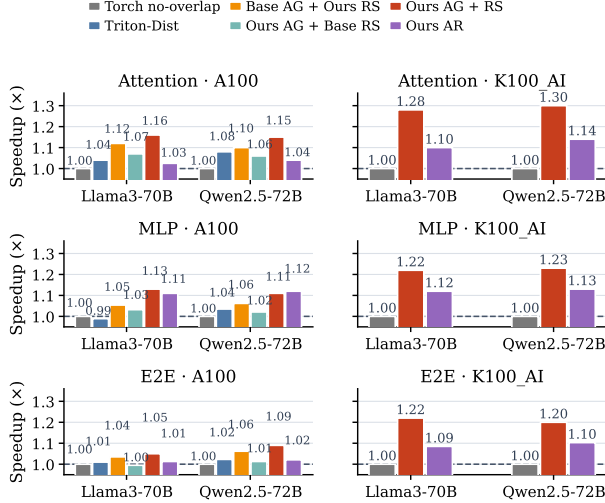


Figure 11. Speedup for TP Attention, TP MLP, and end-to-end prefill.

5.5 Constraint-Aware Configuration Search

Table 3 compares two-stage and exhaustive GEMM-RS search on $(8192, 29568, 8192)$ with 64- and 128-candidate spaces. We measure search cost by the number of search and final-validation measurements, and configuration quality by the maximum chain latency across ranks in final validation. Both methods validate the top three candidates.

For this shape, the two-stage procedure trades at most 1.48% of final latency for an 80.6% and 88.5% reduction in measurement budget.

Table 3. Constraint-aware GEMM-RS search cost and final validation for $8192 \times 29568 \times 8192$.

Candidates	Method	Meas.	Latency	Config
64	Exhaustive	67	5.49 ms	(512,4,4,6,12,2,2,1)
64	Two-stage	13	5.46 ms	(512,4,4,6,16,2,2,1)
128	Exhaustive	131	5.39 ms	(512,4,4,6,24,2,2,1)
128	Two-stage	15	5.47 ms	(512,4,4,6,12,2,2,1)

Note: All searches completed top-3 validation. Configs are listed as *chunk_rows*, *active_chunk_window*, *stage_slots*, *steady_sms*, *tail_sms*, *comm_lanes*, *n_bands*, and *frontier_chunks*.

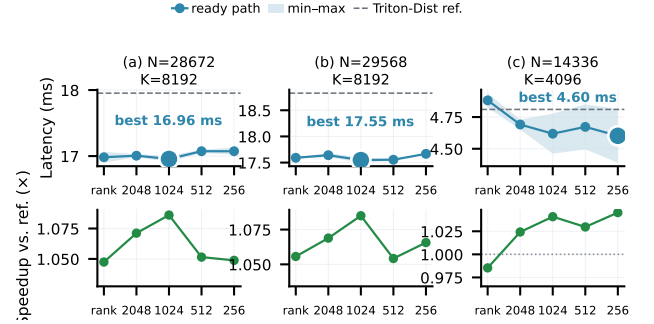


Figure 12. AG-GEMM ready granularity at $M = 28672$ for three (N, K) shapes. Top: rank-max latency and within-shape median reference; bottom: per-cell speedup over its recorded Triton-distributed reference.

6 Conclusion

This paper proposes AnchorOverlap, a runtime execution protocol for collective-GEMM chains. Its core idea is to derive handoff granularity from the minimum data units a consumer needs to start, to use chain-appropriate production and handoff, and to guard one-sided slot reuse with arrival/free tickets.

We implement AG-GEMM, GEMM-RS, and GEMM-AR on two four-accelerator platforms. The protocol reaches up to 1.40 $\times$ operator-level speedup over matched serialized baselines. Cyclic reuse cuts the measured symmetric-staging footprint to as little as one-eighth of full residency, with $L = 4$ halving the footprint within 0.4%–0.6% of median latency, showing that these chains can retain near-full-residency performance without reserving staging for every logical block. AG/RS gains also carry into LM-head-inclusive end-to-end prefill (1.05 $\times$ –1.22 $\times$).

References

- [1] Anirudha Agrawal, Shaizeen Aga, Suchita Pati, and Mahzabeen Islam. 2025. ConCCL: Optimizing ML Concurrent Computation and Communication with GPU DMA Engines. In *2025 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. 1–11. doi:10.1109/ISPASS64960.2025.00018
- [2] Amey Agrawal, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S. Gulavani, and Ramachandran Ramjee. 2023. Sarathi: Efficient LLM Inference by Piggybacking Decodes with Chunked Prefills. *arXiv preprint arXiv:2308.16369* (2023).
- [3] Muhammad Awad, Muhammad Osama, and Brandon Potter. 2025. Iris: First-Class Multi-GPU Programming Experience in Triton. *arXiv preprint arXiv:2511.12500* (2025). <https://arxiv.org/abs/2511.12500>
- [4] Li-Wen Chang, Wenlei Bao, Qi Hou, Chengquan Jiang, Ningxin Zheng, Yinmin Zhong, Xuanrun Zhang, Zuquan Song, Chengji Yao, Ziheng Jiang, Haibin Lin, Xin Jin, and Xin Liu. 2024. FLUX: Fast Software-Based Communication Overlap on GPUs Through Kernel Fusion. *arXiv preprint arXiv:2406.06858* (2024).
- [5] Chang Chen, Xiuhong Li, Qianchao Zhu, Jiangfei Duan, Peng Sun, Xingcheng Zhang, and Chao Yang. 2024. Centauri: Enabling Efficient Scheduling for Communication-Computation Overlap in Large Model Training via Communication Partitioning. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, Vol. 3. 178–191. doi:10.1145/3620666.3651379
- [6] Meghan Cowan, Saeed Maleki, Madanlal Musuvathi, Olli Saarikivi, and Yifan Xiong. 2023. MSCCLang: Microsoft Collective Communication Language. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, Vol. 2. 502–514. doi:10.1145/3575693.3575724
- [7] Minyu Cui, Anna Wingkvist, and Morgan Ericsson. 2026. Fine-grained Computation-Communication Overlap via Tile-level Signaling and Scheduling for Mixture-of-Experts. *arXiv preprint arXiv:2607.19539* (2026). <https://arxiv.org/abs/2607.19539> To appear at ICPP 2026.
- [8] DeepSeek-AI et al. 2026. DeepSeek-V4: Towards Highly Efficient Million-Token Context Intelligence. *arXiv preprint arXiv:2606.19348* (2026).
- [9] Jiarui Fang and Shangchun Zhao. 2024. USP: A Unified Sequence Parallelism Approach for Long Context Generative AI. *arXiv preprint arXiv:2405.07719* (2024).
- [10] Raja Gond, Nipun Kwatra, and Ramachandran Ramjee. 2026. TokenWeave: Efficient Compute-Communication Overlap for Distributed LLM Inference. In *Proceedings of the 9th Conference on Machine Learning and Systems (MLSys)*. arXiv:2505.11329 <https://openreview.net/forum?id=rh2Ylffkq6>
- [11] Ke Hong, Guohao Dai, Jiaming Xu, Qiuli Mao, Xiuhong Li, Jun Liu, Kangdi Chen, Yuhao Dong, and Yu Wang. 2024. FlashDecoding++: Faster Large Language Model Inference with Asynchronization, Flat GEMM Optimization, and Heuristics. In *Proceedings of Machine Learning and Systems*, Vol. 6. 148–161. https://proceedings.mlsys.org/paper_files/paper/2024/hash/5321b1dabcd2be188d796c21b733e8c7-Abstract-Conference.html
- [12] Ke Hong, Xiuhong Li, Minxu Liu, Qiuli Mao, Tianqi Wu, Zixiao Huang, Lufang Chen, Zhong Wang, Yichong Zhang, Zhenhua Zhu, Guohao Dai, and Yu Wang. 2026. Efficient and Adaptable Overlapping for Computation and Communication via Signaling and Reordering. In *Proceedings of the 21st European Conference on Computer Systems (EuroSys)*. 1894–1911. doi:10.1145/3767295.3769370
- [13] Abhinav Jangda, Jun Huang, Guodong Liu, Amir Hosein Nodehi Sabet, Saeed Maleki, Youshan Miao, Madanlal Musuvathi, Todd Mytkowicz, and Olli Saarikivi. 2022. Breaking the Computation and Communication Abstraction Barrier in Distributed Machine Learning Workloads. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. 402–416. doi:10.1145/3503222.3507778
- [14] Ziheng Jiang, Haibin Lin, Yinmin Zhong, Qi Huang, Yanguan Chen, Zhi Zhang, Yanghua Peng, Xiang Li, Cong Xie, Shibiao Nong, Yulu Jia, Sun He, Hongmin Chen, Zhihao Bai, Qi Hou, Shipeng Yan, Ding Zhou, Yiyao Sheng, Zhuo Jiang, Haohan Xu, Haoran Wei, Zhang Zhang, Pengfei Nie, Leqi Zou, Sida Zhao, Liang Xiang, Zherui Liu, Zhe Li, Xiaoying Jia, Jianxi Ye, Xin Jin, and Xin Liu. 2024. MegaScale: Scaling Large Language Model Training to More Than 10,000 GPUs. In *Proceedings of the 21st USENIX Symposium on Networked Systems Design and Implementation (NSDI)*. 745–760. <https://www.usenix.org/conference/nsdi24/presentation/jiang-ziheng>
- [15] Rezaul Karim, Austin Wen, Wang Zongzuo, Weiwei Zhang, Yang Liu, and Walid Ahmed. 2026. CommFuse: Hiding Tail Latency via Communication Decomposition and Fusion for Distributed LLM Training. *arXiv preprint arXiv:2604.24013* (2026).
- [16] Kimi Team et al. 2026. Kimi K3: Open Frontier Intelligence. *arXiv preprint arXiv:2607.24653* (2026). <https://arxiv.org/abs/2607.24653>
- [17] Vijay Anand Korthikanti, Jared Casper, Sangkug Lym, Lawrence McAfee, Michael Andersch, Mohammad Shoeybi, and Bryan Catanzaro. 2022. Reducing Activation Recomputation in Large Transformer Models. *arXiv preprint arXiv:2205.05198* (2022).

- [18] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*. 611–626. doi:10.1145/3600006.3613165
- [19] Seonho Lee, Jihwan Oh, Seokjin Go, and Divya Mahajan. 2025. Characterizing Compute-Communication Overlap in GPU-Accelerated Distributed Deep Learning: Performance and Power Implications. In *Proceedings of the IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. 353–355. doi:10.1109/ISPASS64960.2025.00041
- [20] Shengwei Li, Zhiquan Lai, Dongsheng Li, Yanqi Hao, Weijie Liu, Keshi Ge, Xiaoge Deng, and Kai Lu. 2025. Oases: Efficient Large-Scale Model Training on Commodity Servers via Overlapped and Automated Tensor Model Parallelism. *IEEE Transactions on Parallel and Distributed Systems (TPDS)* 36, 9 (2025), 1828–1840. doi:10.1109/TPDS.2025.3583165
- [21] Hao Liu, Matei Zaharia, and Pieter Abbeel. 2024. Ring Attention with Blockwise Transformers for Near-Infinite Context. In *Proceedings of the 12th International Conference on Learning Representations (ICLR)*. <https://openreview.net/forum?id=WsRHpHH4s0>
- [22] Deepak Narayanan, Aaron Harlap, Amar Phanishayee, Vivek Seshadri, Nikhil R. Devanur, Gregory R. Ganger, Phillip B. Gibbons, and Matei Zaharia. 2019. PipeDream: Generalized Pipeline Parallelism for DNN Training. In *Proceedings of the 27th Symposium on Operating Systems Principles (SOSP)*. 1–15. arXiv:1806.03377 doi:10.1145/3341301.3359646
- [23] Deepak Narayanan, Mohammad Shoeybi, Jared Casper, Patrick LeGresley, Mostofa Patwary, Vijay Anand Korthikanti, Dmitri Vainbrand, Prethvi Kashinkunti, Julie Bernauer, Bryan Catanzaro, Amar Phanishayee, and Matei Zaharia. 2021. Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*. 1–15. doi:10.1145/3458817.3476209
- [24] NVIDIA. 2024. cuBLASmp: Parallel GEMM and Communication. <https://docs.nvidia.com/cuda/cublasmp/>.
- [25] NVIDIA. 2024. NCCL: NVIDIA Collective Communication Library. <https://github.com/NVIDIA/nccl>.
- [26] NVIDIA. 2024. NVSHMEM: OpenSHMEM-based PGAS for GPU. <https://developer.nvidia.com/nvshmem>.
- [27] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Ínigo Goiri, Saeed Maleki, and Ricardo Bianchini. 2024. Splitwise: Efficient Generative LLM Inference Using Phase Splitting. In *Proceedings of the 51st Annual International Symposium on Computer Architecture (ISCA)*. 118–132. doi:10.1109/ISCA59077.2024.00019
- [28] Suchita Pati, Shaizeen Aga, Mahzabeen Islam, Nuwan Jayasena, and Matthew D. Sinclair. 2024. T3: Transparent Tracking & Triggering for Fine-grained Overlap of Compute & Collectives. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, Vol. 2. 1146–1164. doi:10.1145/3620665.3640410
- [29] Ramya Prabhu, Ajay Nayak, Jayashree Mohan, Ramachandran Ramjee, and Ashish Panwar. 2025. vAttention: Dynamic Memory Management for Serving LLMs without PagedAttention. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. 1133–1150. doi:10.1145/3669940.3707256
- [30] PyTorch Foundation and TorchTitan Team. 2024. Async Tensor Parallelism (Async-TP). <https://discuss.pytorch.org/t/distributed-w-torchtitan-introducing-async-tensor-parallelism-in-pytorch/209487>.
- [31] Xinwei Qiang, Yue Guan, Zhengding Hu, Keren Zhou, Yufei Ding, and Adnan Aziz. 2026. Syncopate: Efficient Multi-GPU AI Kernels via Automatic Chunk-Centric Compute-Communication Overlap. In *Proceedings of the 20th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. USENIX Association, Seattle, WA, 331–347. arXiv:2601.20595 <https://www.usenix.org/conference/osdi26/presentation/qiang>
- [32] Qwen Team. 2026. Qwen3.8-Max: A New Bar for Coding and Cowork. <https://qwen.ai/blog?id=qwen3.8>
- [33] Aashaka Shah, Vijay Chidambaram, Meghan Cowan, Saeed Maleki, Madan Musuvathi, Todd Mytkowicz, Jacob Nelson, Olli Saarikivi, and Rachee Singh. 2023. TACCL: Guiding Collective Algorithm Synthesis using Communication Sketches. In *Proceedings of the 20th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*. 593–612. <https://www.usenix.org/conference/nsdi23/presentation/shah>
- [34] Baodi Shan, Mauricio Araya-Polo, and Barbara Chapman. 2026. GICC: A High-Performance Runtime for GPU-Initiated Communication and Coordination in Modern HPC Systems. In *Proceedings of the 35th International Symposium on High-Performance Parallel and Distributed Computing (HPDC)*. 45–57. doi:10.1145/3806645.3807576
- [35] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. 2019. Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism. *arXiv preprint arXiv:1909.08053* (2019).
- [36] Weigao Sun, Zhen Qin, Weixuan Sun, Shidi Li, Dong Li, Xuyang Shen, Yu Qiao, and Yiran Zhong. 2024. CO2: Efficient Distributed Training with Full Communication-Computation Overlap. In *Proceedings of the 12th International Conference on Learning Representations (ICLR)*. <https://openreview.net/forum?id=ZO5cn4lfaN>

- [37] Guanhua Wang, Shivaram Venkataraman, Amar Phanishayee, Jorgen Thelin, Nikhil Devanur, and Ion Stoica. 2020. Blink: Fast and Generic Collectives for Distributed ML. In *Proceedings of Machine Learning and Systems*, Vol. 2. 172–186. https://proceedings.mlsys.org/paper_files/paper/2020/hash/cd3a9a55f7f3723133fa4a13628cdf03-Abstract.html
- [38] Guanhua Wang, Chengming Zhang, Zheyu Shen, Ang Li, and Olatunji Ruwase. 2024. Domino: Eliminating Communication in LLM Training via Generic Tensor Slicing and Overlapping. *arXiv preprint arXiv:2409.15241* (2024).
- [39] Shibo Wang, Jinliang Wei, Amit Sabne, Andy Davis, Berkin Ilbeyi, Blake Hechtman, Dehao Chen, Karthik Srinivasa Murthy, Marcello Maggioni, Qiao Zhang, Sameer Kumar, Tongfei Guo, Yuanzhong Xu, and Zongwei Zhou. 2023. Overlap Communication with Dependent Computation via Decomposition in Large Deep Learning Models. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, Vol. 1. 93–106. doi:10.1145/3567955.3567959
- [40] Jianwen Xian, Zhiyuan Xu, Yuchen Li, Ziliang Lai, Kang He, Zhen Huang, Aichen Feng, Jinyan Chen, Yilin Zhang, Qinqin Chen, and Chengru Song. 2026. X-Stage: An Overlooked Pipeline Stage for Communication-Computation Overlap in DiT Inference. *arXiv preprint arXiv:2607.23264* (2026). <https://arxiv.org/abs/2607.23264>
- [41] Bin Xiao and Lei Su. 2024. ISO: Overlap of Computation and Communication within Sequence For LLM Inference. *arXiv preprint arXiv:2409.11155* (2024).
- [42] Guanbin Xu, Zhihao Le, Yinhe Chen, Zhiqi Lin, Zewen Jin, Youshan Miao, and Cheng Li. 2025. AutoCCL: Automated Collective Communication Tuning for Accelerating Distributed and Parallel DNN Training. In *Proceedings of the 22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI)*. 667–683. <https://www.usenix.org/conference/nsdi25/presentation/xu-guanbin>
- [43] Shulai Zhang, Ningxin Zheng, Haibin Lin, Ziheng Jiang, Wenlei Bao, Chengquan Jiang, Qi Hou, Weihao Cui, Size Zheng, Li-Wen Chang, Quan Chen, and Xin Liu. 2025. Comet: Fine-grained Computation-communication Overlapping for Mixture-of-Experts. In *Proceedings of the 8th Conference on Machine Learning and Systems (MLSys)*. <https://openreview.net/forum?id=fGgQS5VW09>
- [44] Size Zheng, Wenlei Bao, Qi Hou, Xuegui Zheng, Jin Fang, Chenhui Huang, Tianqi Li, Haojie Duanmu, Renze Chen, Ruifan Xu, Yifan Guo, Ningxin Zheng, Ziheng Jiang, Xinyi Di, Dongyang Wang, Jianxi Ye, Haibin Lin, Li-Wen Chang, Liqiang Lu, Yun Liang, Jidong Zhai, and Xin Liu. 2025. Triton-distributed: Programming Overlapping Kernels on Distributed AI Systems with the Triton Compiler. *arXiv preprint arXiv:2504.19442* (2025).
- [45] Size Zheng, Jin Fang, Xuegui Zheng, Qi Hou, Wenlei Bao, Ningxin Zheng, Ziheng Jiang, Dongyang Wang, Jianxi Ye, Haibin Lin, Li-Wen Chang, and Xin Liu. 2025. TileLink: Generating Efficient Compute-Communication Overlapping Kernels using Tile-Centric Primitives. In *Proceedings of the Eighth Conference on Machine Learning and Systems (MLSys)*. <https://openreview.net/forum?id=ccjvBkTRRe>
- [46] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. 2024. DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving. In *Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. <https://www.usenix.org/conference/osdi24/presentation/zhong-yinmin>